

STAT 4830 NAPPERS Project

Final Deliverable and Reproducibility Guide

Naseebullah Andar, Samantha Agisim, Patrick Ledoit

1 Dynamic Portfolio Optimization in Equities and Prediction Markets

This repository contains our final STAT 4830 project, *Dynamic Portfolio Optimization in Equities and Prediction Markets*. The project studies portfolio construction as a numerical optimization problem in two settings: a time-varying universe of U.S. equities and a Polymarket-style panel of prediction-market contracts. The equity portion provides the original optimization framework. It implements a dynamic long-only mean-variance optimizer with rolling estimates of expected returns and covariance risk, turnover penalties, PyTorch-based gradient optimization, monthly and daily backtests, and benchmark comparisons. The prediction-market portion extends the same allocation logic to contracts whose prices can be interpreted as market-implied probabilities. In that setting, the project studies mispricing, executable prices, liquidity constraints, event-level concentration, chronology-safe validation, and unresolved-market limitations. The common theme across both parts is allocation under uncertainty: given noisy estimates, market prices, constraints, and imperfect data, how should capital be allocated, and how should the resulting optimization pipeline be evaluated?

2 Main Question

Can a constrained numerical optimization framework be adapted from equity portfolios to prediction-market contracts, and what does the resulting pipeline reveal about calibration, predictability, liquidity, execution assumptions, and validation design? The goal is to build a reproducible research pipeline and evaluate where the optimization framework appears informative, fragile, or limited by the available data.

3 Final Deliverable Folder

The polished submission materials are located in:

```
final_deliverable/
```

The folder is organized as follows:

```
final_deliverable/
|-- final_report/
|   '-- Dynamic_Portfolio_Optimization_in_Equities_and_Prediction_Markets.pdf
|
|-- implementation_code/
|   |-- demo_01_standalone_optimization_equity_and_polymarket.ipynb
|   |-- demo_02_standalone_polymarket_panel_experiments.ipynb
|   |-- STAT_4830_Portfolio_Optimization_Daily_Regime.ipynb
|   '-- STAT_4830_Portfolio_Optimization_Monthly_Regime.ipynb
|
|-- llm_exploration/
|   '-- STAT_4830_LLM_Logs.pdf
|
|-- readme/
|   '-- README.tex
```

```
|  
|-- self_critiques/  
    |-- STAT_4830_Self_Critique_Edited.pdf
```

The `final_deliverable` folder is the main folder to review for the assignment. It contains the final report, implementation notebooks, LLM exploration logs, self-critique, and this README file.

4 Main Deliverables

- `final_report/`

Contains the final written report. This is the main paper for the project. It describes the research question, data sources, optimization framework, equity results, prediction-market extension, limitations, and conclusions.

- `implementation_code/`

Contains the notebooks used to implement and reproduce the main experiments. These include the monthly equity optimizer, daily equity extension, standalone equity and Polymarket demo, and prediction-market panel experiments.

- `llm_exploration/`

Contains the LLM exploration log. This document explains how LLMs were used throughout the project for ideation, code scaffolding, debugging, data inspection, writing, and presentation preparation, while distinguishing LLM assistance from our own modeling and research decisions.

- `self_critiques/`

Contains the final self-critique. This document reflects on project scope, validation choices, limitations, LLM use, and future research directions.

- `readme/`

Contains this reproducibility guide.

5 Broader Repository Structure

The broader repository contains data, source code, scripts, notebooks, figures, tests, and development materials used throughout the semester.

```
STAT-4830-NAPPERS-project/
|-- data/
|   |-- other/
|   |-- prediction_market/
|       |-- Kalshi_Data
|       |-- polymarket_daily_panel_60plus.csv
|       |-- polymarket_markets_rich.csv
|       |-- polymarket_time_data.zip
|   '-- stock_market/
|
|-- docs/
|   |-- assignments/
|   |-- llm_exploration/
|   '-- polymarket_panel/
|
|-- figures/
|   '-- Images/
|
|-- final_deliverable/
|
|-- notebooks/
|   |-- demo_01_optimization_equity_and_polymarket.ipynb
|   |-- demo_01_standalone_optimization_equity_and_polymarket.ipynb
```

```

| |-- demo_02_polymarket_panel_experiments.ipynb
| |-- demo_02_standalone_polymarket_panel_experiments.ipynb
| |-- STAT_4830_Portfolio_Optimization_Daily_Regime.ipynb
| |-- STAT_4830_Portfolio_Optimization_Monthly_Regime.ipynb
| '-- Week*.ipynb
|
|-- other/
| |-- polymarket_exploration.ipynb
| |-- polymarket_exploration_60plus.ipynb
| '-- polymarket_time_data.py
|
|-- scripts/
| |-- run_panel_experiments.py
| '-- run_polymarket_mispricing.py
|
|-- src/
| |-- polymarket/
| |-- data_loader.py
| |-- features.py
| |-- model.py
| '-- utils.py
|
|-- supplementary class setup material/
|
|-- tests/
| '-- test_basic.py
|
|-- README.md
|-- pyproject.toml

```

```
'-- uv.lock
```

The `Week*.ipynb` notebooks and other exploratory notebooks document earlier development work. For the final assignment, the most important materials are the files in `final_deliverable/`. For reproduction and development history, the broader repository folders may also be useful.

6 Environment Setup

Use Python 3.10 or newer. From the repository root:

```
python -m venv .venv
source .venv/bin/activate      # macOS/Linux
# .venv\Scripts\activate      # Windows

python -m pip install --upgrade pip
python -m pip install numpy pandas matplotlib scikit-learn torch scipy jupyter
    nbformat nbclient pytest
```

Run the basic tests:

```
python -m pytest -q
```

When running scripts from the command line, set the import path:

```
export PYTHONPATH=src          # macOS/Linux
# $env:PYTHONPATH="src"        # Windows PowerShell
```

7 Recommended Reading Order

For the final assignment, we recommend reviewing the materials in this order:

1. Read the final report:

```
final_deliverable/final_report/  
Dynamic_Portfolio_Optimization_in_Equities_and_Prediction_Markets.pdf
```

2. Review the standalone implementation notebook:

```
final_deliverable/implementation_code/  
demo_01_standalone_optimization_equity_and_polymarket.ipynb
```

3. Review the monthly and daily equity notebooks:

```
final_deliverable/implementation_code/  
STAT_4830_Portfolio_Optimization_Monthly_Regime.ipynb  
  
final_deliverable/implementation_code/  
STAT_4830_Portfolio_Optimization_Daily_Regime.ipynb
```

4. Review the prediction-market panel notebook:

```
final_deliverable/implementation_code/  
demo_02_standalone_polymarket_panel_experiments.ipynb
```

5. Read the LLM logs:

```
final_deliverable/llm_exploration/  
STAT_4830_LLM_Logs.pdf
```

6. Read the self-critique:

```
final_deliverable/self_critiques/  
STAT_4830_Self_Critique_Edited.pdf
```

8 Fast Reproduction Path

For a quick reproduction of the main project logic, run the standalone notebooks in

```
final_deliverable/implementation_code/:
```

```
jupyter notebook final_deliverable/implementation_code/  
    demo_01_standalone_optimization_equity_and_polymarket.ipynb  
  
jupyter notebook final_deliverable/implementation_code/  
    demo_02_standalone_polymarket_panel_experiments.ipynb
```

The first demo notebook gives a compact overview of the equity and Polymarket optimization logic. The second demo notebook focuses on prediction-market panel experiments.

9 Full Reproduction Path

A fuller reproduction from the repository root is:

```
# 1. Create environment  
python -m venv .venv  
source .venv/bin/activate  
  
# 2. Install dependencies  
python -m pip install --upgrade pip  
python -m pip install numpy pandas matplotlib scikit-learn torch scipy jupyter  
    nbformat nbclient pytest  
  
# 3. Run tests  
python -m pytest -q
```

```

# 4. Set import path
export PYTHONPATH=src

# 5. Run prediction-market panel experiments
python scripts/run_panel_experiments.py \
    --csv data/prediction_market/polymarket_daily_panel_60plus.csv \
    --out Outputs/polymarket_panel_experiments

# 6. Optional: run Polymarket mispricing optimizer
python scripts/run_polymarket_mispricing.py \
    --csv data/prediction_market/polymarket_daily_panel_60plus.csv \
    --out Outputs/polymarket_mispricing

# 7. Run final deliverable notebooks
jupyter notebook final_deliverable/implementation_code/
    demo_01_standalone_optimization_equity_and_polymarket.ipynb
jupyter notebook final_deliverable/implementation_code/
    demo_02_standalone_polymarket_panel_experiments.ipynb
jupyter notebook final_deliverable/implementation_code/
    STAT_4830_Portfolio_Optimization_Monthly_Regime.ipynb
jupyter notebook final_deliverable/implementation_code/
    STAT_4830_Portfolio_Optimization_Daily_Regime.ipynb

```

For a shorter check, run only steps 1–4 and the two standalone demo notebooks.

10 Equity Optimization

The equity optimizer is the original framework that motivated the project. It studies whether a rolling, risk-aware, turnover-aware optimizer can improve on equal weighting over the same

dynamic universe.

Core objective:

```
maximize_w mu_t' w - gamma * w' Sigma_t w - kappa * ||w - w_prev||_1
subject to w >= 0, sum(w) = 1
```

The main equity notebooks are:

```
final_deliverable/implementation_code/
STAT_4830_Portfolio_Optimization_Monthly_Regime.ipynb

final_deliverable/implementation_code/
STAT_4830_Portfolio_Optimization_Daily_Regime.ipynb
```

The monthly notebook implements the rolling monthly equity optimizer, sensitivity analysis, and benchmark comparisons. The daily notebook adapts the same logic to a higher-frequency setting using frequency-consistent rolling windows, history requirements, and turnover scaling.

11 Prediction-Market Mispricing Optimizer

The prediction-market extension adapts the equity allocation framework to Polymarket-style contracts. Instead of estimating stock returns, the model estimates whether a contract appears mispriced relative to an executable market price.

For each contract:

```
edge_i = p_hat_i - c_exec_i
```

where p_hat_i is the model-estimated probability and c_exec_i is the estimated executable Yes price.

The optimizer maximizes:

```
F(w) = sum_i w_i * edge_i
```

```
- gamma * sum_i w_i^2 * p_hat_i * (1 - p_hat_i)
- kappa * ||w - w_prev||_1
```

subject to nonnegative long-Yes positions, cash being allowed, gross exposure caps, per-contract caps, per-event caps, and liquidity-based position limits.

The main prediction-market notebook is:

```
final_deliverable/implementation_code/
demo_02_standalone_polymarket_panel_experiments.ipynb
```

The modular code is in:

```
src/polymarket/
```

and the main command-line scripts are:

```
scripts/run_polymarket_mispricing.py
scripts/run_panel_experiments.py
```

12 Data

The project uses two broad classes of data.

Equity Data

The equity data are stored under:

```
data/stock_market/
```

These files support the monthly equity portfolio optimization experiments. The daily equity return dataset used for the daily-regime extension is too large to include directly in the GitHub repository. Readers who would like access to the daily equity data for reproduction purposes should contact the authors at:

andar@sas.upenn.edu

The daily equity notebook documents the required data structure and can be run once the corresponding daily return file is made available locally.

Prediction-Market Data

The prediction-market data are stored under:

```
data/prediction_market/
```

This folder includes:

```
Kalshi_Data  
polymarket_daily_panel_60plus.csv  
polymarket_markets_rich.csv  
polymarket_time_data.zip
```

The prediction-market pipeline relies primarily on the Polymarket-style data files. A key limitation is that the packaged artifact does not include final contract resolutions for all contracts, so prediction-market realized PnL is evaluated using next-step quote-to-quote changes rather than final settlement outcomes.

13 Expected Outputs

Depending on which notebooks and scripts are run, outputs may be written to folders such as:

```
Outputs/polymarket_mispricing/  
Outputs/polymarket_panel_experiments/  
Outputs/demo_optimization/  
Outputs/demo_polymarket_panel_experiments/
```

Expected outputs include performance summaries, plots, allocation diagnostics, weight files, turnover diagnostics, and prediction-market panel reports.

14 Interpretation of Results

The equity results show that the optimization framework works mechanically and can materially outperform equal weighting after coherent recalibration. They also show that portfolio optimization is highly sensitive to noisy estimates, regularization, rolling-window design, turnover penalties, and solver settings. The prediction-market results are best interpreted as a baseline research extension rather than a final trading system. They show that the optimization framework can be transported to a setting where prices represent market-implied probabilities. However, they also show that positive objective values and disciplined allocations do not automatically translate into robust realized PnL when data are incomplete and outcomes are unresolved. The main takeaway is not that we found a guaranteed trading strategy. The main takeaway is that portfolio optimization must be evaluated as a full empirical pipeline, including signals, constraints, execution assumptions, liquidity, chronology-safe validation, and realistic performance measurement.

15 Development Process and LLM Use

The file

`final_deliverable/llm_exploration/STAT_4830_LLM_Logs.pdf`

documents our use of LLMs throughout the project. LLMs helped with brainstorming, code scaffolding, debugging, data inspection, explanation, writing, and presentation preparation. However, the team remained responsible for choosing the research direction, validating assumptions, running experiments, checking code, interpreting outputs, and deciding which claims were supported by the evidence.

This distinction is important because the project used LLMs as technical assistants, not as automatic research engines.

16 Self-Critique and Limitations

The file

`final_deliverable/self_critiques/STAT_4830_Self_Critique_Edited.pdf`

provides a detailed reflection on the strengths and limitations of the project.

The main limitations are:

- The equity optimizer relies heavily on rolling sample means and covariance estimates, which are noisy in financial data.
- Hyperparameter exploration was informative, but future work with more date-level observations could impose an even cleaner development, validation, and test split.
- Equal-weight and broad index benchmarks are useful, but future work should include additional internal baselines such as market-cap weighting, minimum variance, risk parity, momentum, and volatility-scaled strategies.
- The prediction-market extension is limited by the available data. The packaged artifact does not include fully resolved outcomes for all contracts, so realized PnL is measured using next-step quote changes rather than final settlement values.
- The prediction-market model is intentionally simple and should be viewed as a baseline research framework rather than a final trading system.

17 Future Research Directions

Several natural extensions follow from the project:

- Use shrinkage covariance estimators, factor-model covariance matrices, Bayesian expected-return estimates, and stronger regularization in the equity optimizer.
- Add more portfolio-rule baselines, including market-cap weighting, minimum variance, risk parity, momentum, and volatility-scaled strategies.
- Build a resolved prediction-market dataset with contract histories, order-book data, liquidity measures, event categories, final resolutions, and settlement dates.
- Evaluate prediction-market performance using settlement-based PnL rather than only next-step quote-to-quote changes.
- Add richer prediction-market features, including text embeddings, event categories, price histories, volume changes, order-book depth, news signals, and external fundamentals.
- Make the prediction-market optimizer more execution-aware by modeling bid-ask spreads, market depth, partial fills, slippage, fees, and order timing.
- Modularize the code further into a more reusable research package.

18 Troubleshooting

`ModuleNotFoundError: No module named 'polymarket'`

Set the Python path before running scripts:

```
export PYTHONPATH=src
```

Tests Cannot Import Local Modules

Run:

```
python -m pytest -q
```

instead of bare `pytest`.

Output Folder Issues

Use `Outputs/`, not `outputs/`. The capitalization matters on case-sensitive systems.

Older Notebooks Ask for Manual Upload

Use the final deliverable notebooks or the standalone demo notebooks for reproduction. The older weekly notebooks are kept as development history and may contain Colab-specific upload cells.

19 Project Status

This repository contains a reproducible research pipeline for dynamic portfolio optimization in equities and prediction markets. The equity portion provides the original optimization base. The prediction-market portion extends the same allocation logic to a newer and less standardized market setting. The project should be read as an applied numerical optimization study. Its main contribution is to show how optimization objectives, constraints, numerical solvers, regularization, data limitations, and evaluation design interact when theory is turned into an empirical allocation system.

20 Authors and Course Information

- Naseebullah Andar
- Samantha Agisim
- Patrick Ledoit

Course: STAT 4830: Numerical Optimization for Data Science and Machine Learning

Research Advisor: Dr. Damek Davis